

Lab 2 Assignment (CSI3120)

Functional Programming

Objective

The objective of this lab is to implement a job scheduling system in OCaml. You will learn how to gather input from the terminal, process the input, and create various scheduling strategies such as minimizing idle time, avoiding overlaps, or maximizing job priority. This lab will help you practice basic functional programming techniques, recursion, list manipulation, and working with terminal input and output.

Instructions

1. You are tasked with creating a job scheduling system in OCaml. Each job has a start time (in hours and minutes), a duration (in minutes), and a priority. You will gather this information from the terminal for each job.
2. You will implement different scheduling strategies:
 - No Overlaps: Ensures jobs are scheduled without overlaps.
 - Maximize Priority: Schedules jobs with the highest priority first.
 - Minimize Idle Time: Schedules jobs to minimize gaps between them.
3. The input for this lab will be received from the terminal, so you will write code that asks the user to enter details for each job (start time, duration, and priority) and then choose a scheduling strategy.

Examples of Input and Output

Example 1:

How many jobs do you want to schedule? 2

For job 1, please enter the following details:

- Start Time (hours): 9
- Start Time (minutes): 30
- Duration (minutes): 60
- Priority: 3

For job 2, please enter the following details:

- Start Time (hours): 11
- Start Time (minutes): 0

- Duration (minutes): 45
- Priority: 5

Choose a scheduling strategy (1 for No Overlaps, 2 for Max Priority, 3 for Minimize Idle Time): 1

Scheduled Jobs (No Overlaps):

Job scheduled: Start Time = 570 minutes, Duration = 60 minutes, Priority = 3

Job scheduled: Start Time = 660 minutes, Duration = 45 minutes, Priority = 5

Step-by-Step Guide

Here is a step-by-step guide and the necessary functions to complete the assignment:

Step 1: Define Job Representation

Define a `job` type that includes three fields: `start_time`, `duration`, and `priority`. This structure will be used to store information about each job.

Step 2: Convert Time to Minutes

Write a helper function to convert the time (hours and minutes) to total minutes from midnight. This will simplify your calculations. (name it `time_to_minutes`)

Step 3: Reading Input from the Terminal

Write a function `read_jobs` that will prompt the user to input job details. This function should prompt for each job's start time (hours and minutes), duration (in minutes), and priority. Use `Printf.printf` to display the prompts, and `input_line` to read input from the terminal. Make sure to trim the input to avoid errors.

Step 4: Implement Scheduling Functions

Write three functions to handle different scheduling strategies:

- `schedule_jobs`: Ensures no job overlaps.
- `schedule_jobs_max_priority`: Prioritizes jobs with higher priority.
- `schedule_jobs_min_idle`: Minimizes idle time between jobs.

Step 5: Print Scheduled Jobs

Implement a function `print_schedule` to display the scheduled jobs in a user-friendly format, including their start time, duration, and priority.

Step 6: Main Function and Terminal Input

Write a main function to bring everything together. This function should prompt the user to enter how many jobs they want to schedule, gather input for each job, and ask them to select a scheduling strategy. Then, the program should schedule and display the jobs according to the chosen strategy.

Lab Submission Guidelines

The due date for each lab is two weeks from the date of the lab session. For Lab 2, the submission deadline is **October 4th**. Each group is required to make a single submission on Brightspace as two separate files:

1. One code file written in OCaml.
2. One PDF report explains your coding concisely and includes a few screenshots of your output using different inputs.

On the front page of your report, please include:

- Group number
- Names of all group members
- Student IDs

You will be evaluated as a group unless one of the members raises concerns about unequal contributions. In such cases, we may ask for further clarification and reserve the right to evaluate members individually.

If you use or receive help from ChatGPT or similar tools, you must clearly reference it in your submission. Failure to do so will result in a **zero** for the lab, and further actions may be taken in accordance with university policies.

You are allowed to use tools like ChatGPT for assistance, but **copying and pasting** the generated results directly into your assignment will not be tolerated. You are expected to **understand the help provided and implement the solution in your own words and approach**.